

# Kiro + Amazon DocumentDB

## Complete Study Guide

Build a Fitness Center Management App from idea to running MVP in under 2 hours using spec-driven AI development. Every step explained simply, with diagrams and code.

Python Flask

Amazon DocumentDB

Kiro IDE

TLS / SSH Tunnel

MVP in 2 hrs

## 0 What Are We Building & Why?

The big picture before we dive into code

Imagine a gym that still uses paper forms to track client workouts, body measurements, and progress. Every time a trainer needs to see a client's history, they flip through folders. It's slow, error-prone, and impossible to analyze. We're going to digitize all of this — and build a real web app to do it — in under 2 hours. How? By using Kiro IDE and Amazon DocumentDB together.

### 🔗 SIMPLE ANALOGY

Think of Kiro like a very smart architect who listens to your idea, draws the blueprint, plans the construction schedule, and then also does the actual building — all automatically. And DocumentDB is like a filing cabinet that can store messy, irregular fitness data the way it actually exists, not the way a rigid spreadsheet wants it.

### 📁 Kiro IDE

- ✓ Spec-driven: you describe, it builds
- ✓ EARS format requirements (structured user stories)
- ✓ Auto-generates architecture design docs
- ✓ Test-driven — tests run after every task
- ✓ Smart Hooks for workflow automation
- ✓ Idea → working MVP in < 2 hours

### 📁 Amazon DocumentDB

- ✓ Stores data as flexible JSON documents
- ✓ MongoDB-compatible (use PyMongo to connect)
- ✓ Perfect for fitness data — no rigid schema
- ✓ Nested structures (exercises inside workouts)
- ✓ Aggregation pipeline for real-time analytics
- ✓ Fully managed, serverless, scales automatically

## Why NOT a regular SQL database?

SQL databases (like MySQL, PostgreSQL) need a fixed schema. Every client must have exactly the same fields. But fitness data is messy:

- One client tracks: weight, basic cardio, steps
- Another tracks: VO2 max, heart rate variability, muscle-specific strength assessments
- Each workout program has different exercises, sets, reps, weights, rest periods

With SQL you'd need 10+ tables and complex JOINS. With DocumentDB, each client is just one JSON document — flexible and natural.

## 🧩 Key Concepts Explained Simply

Terms you'll hear in the workshop — explained like you're learning for the first time

### 🔑 SPEC-DRIVEN DEVELOPMENT

**What it means:** Instead of coding immediately, you first write a specification — a document that clearly defines what the software should do. Only after the spec is approved does coding begin.

**Why it matters:** Most bugs happen because developers built the wrong thing. A spec prevents this by forcing clarity first.

#### ANALOGY

An architect draws blueprints before construction begins. Spec-driven development is the same — blueprint first, then build.

### 🔑 EARS FORMAT

**Full name:** Easy Approach to Requirements Syntax

**Pattern:** WHEN [trigger] THE SYSTEM SHALL [action]

```
"WHEN a trainer submits client details,
THE SYSTEM SHALL create a profile with
personal info, goals, and health notes."
```

This format makes requirements unambiguous — no room for guesswork.

### 🔑 TEST-DRIVEN DEVELOPMENT (TDD)

**What it means:** Tests are written (or auto-generated by Kiro) before the code. After writing code, tests run immediately to verify everything works.

**Kiro's approach:** After completing each task, Kiro automatically runs tests. If they fail, it fixes the code. Only when tests pass does it move to the next task.

### 🔑 MVP (MINIMUM VIABLE PRODUCT)

**What it means:** The smallest version of a product that works and delivers value. Only the core features — nothing extra.

**Our MVP vs Full Plan:**

| PLAN      | TASKS            | TIME      |
|-----------|------------------|-----------|
| Full Plan | 15 main + 22 sub | Days      |
| MVP Plan  | 8 focused tasks  | < 2 hours |

### 🔑 SSH TUNNEL

**Problem:** DocumentDB lives inside AWS VPC (Virtual Private Cloud) — it has no public internet address. You can't connect directly from your laptop.

**Solution:** SSH Tunnel. You connect to an EC2 instance that IS publicly accessible, and that EC2 instance forwards your connection to DocumentDB.

```
Your laptop → EC2 bastion → DocumentDB
```

### 🔑 TLS CERTIFICATE (PEM FILE)

**What it is:** A security certificate that proves you're connecting to the real DocumentDB and not a fake server. It encrypts all data in transit.

**File name:** `global-bundle.pem`

**Without it:** Connection will be refused. Always download it first.

## 0 Prerequisites & Setup

Everything you need before starting — do this once, then forget about it

### The Exact Commands

TERMINAL — RUN THESE IN ORDER

```
# STEP 1: Install Python dependencies
pip install flask pymongo dnspython

# STEP 2: Download SSL Certificate (run from your project folder)
wget https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem

# STEP 3: Open the SSH Tunnel (replace with your actual values)
# Leave this terminal running — don't close it!
ssh -L 27017:your-docdb-endpoint:27017 ec2-user@your-ec2-bastion -N

# STEP 4: Run the finished app (after building with Kiro)
python run.py
```

#### ⚠ COMMON MISTAKES

1. Forgetting to start the SSH tunnel before running the app — you'll get "connection refused" error.
2. Forgetting to download global-bundle.pem — you'll get "SSL handshake failed".
3. Closing the SSH tunnel terminal while the app is running — connection will drop.

### 1 INSTALL KIRO IDE

Download from [kiro.dev/downloads](https://kiro.dev/downloads). Available for macOS, Windows, and Linux. It looks similar to VS Code — tabbed editor, file explorer, integrated terminal.

### 2 CREATE DOCUMENTDB CLUSTER

Go to AWS Console → DocumentDB → Create cluster. Note the cluster endpoint. It'll look like:  
cluster.cluster-xxxx.us-east-1.docdb.amazonaws.com

### 3 SET UP SSH TUNNEL

This creates the bridge from your local machine into the AWS VPC. Run this command and leave the terminal open while working.

### 4 DOWNLOAD SSL CERTIFICATE

One command downloads the security certificate. Without this, DocumentDB will reject your connection.

## How the Connection Works (Visualized)

NETWORK FLOW — LOCAL MACHINE TO DOCUMENTDB



# 1 Step 1 — Starting with Kiro Specs

From rough idea to structured requirements — without writing a single line of code yet

This is the most important step and what makes Kiro fundamentally different. Instead of immediately coding, Kiro asks: "What exactly are you building?" You write a plain English prompt. Kiro reads it and generates a formal requirements document.

## The Prompt You Give to Kiro

KIRO CHAT INPUT — PASTE THIS EXACTLY

```
I need to build a Fitness Center management system that digitizes paper-based fitness tracking using Python as the primary development language.
```

```
Application needs to be a web application.
```

```
The system should use Amazon DocumentDB as the backend database to handle complex workout data structures and support analytics queries.
```

```
Key requirements include:
```

- Client registration
- Workout program creation and management
- Body measurement recording and trends
- Client progress tracking and analytics

### 🔗 WHAT TO SPECIFY VS WHAT TO LEAVE TO KIRO

**You specify:** Python (language), DocumentDB (database), web application (type)

**Leave to Kiro:** Flask (framework), Bootstrap (UI), folder structure, file naming. Kiro picks the best options.

## What Kiro Does After You Send the Prompt



## The 6 EARS Requirements Kiro Generates

Kiro automatically creates these structured requirements. Read them carefully before approving — this is your chance to catch anything missing!

#### REQ-1 · Client Registration

**WHEN a trainer submits client details, THE SYSTEM SHALL create a profile with personal information, goals, and health notes.**

##### ACCEPTANCE CRITERIA

- System stores name, email, phone, date of birth
- System stores fitness goals and health notes
- Duplicate email addresses are rejected
- Profile is immediately viewable after creation

#### REQ-2 · Workout Management

**WHEN a trainer creates a workout program, THE SYSTEM SHALL store structured exercise data with sets, reps, weights, and rest periods for each exercise.**

##### ACCEPTANCE CRITERIA

- Multiple exercises can be added to one program
- Each exercise stores: name, sets, reps, weight (kg), rest (seconds)
- Programs are linked to a specific client

#### REQ-3 · Body Measurements

**WHEN measurements are recorded, THE SYSTEM SHALL calculate trends and flag significant changes over time.**

##### ACCEPTANCE CRITERIA

- Records: weight, height, BMI, body fat percentage
- BMI is auto-calculated from weight and height
- Significant changes ( $\pm 5\%$ ) are highlighted

#### REQ-4 · Progress Analytics

**THE SYSTEM SHALL provide real-time dashboards showing client performance and goal achievement.**

#### REQ-5 · Authentication

**WHEN a user logs in, THE SYSTEM SHALL verify credentials and assign role-based access (trainer vs admin).**

#### REQ-6 · Reporting

**THE SYSTEM SHALL generate printable progress reports per client covering selected time ranges.**

#### IMPORTANT — WHAT YOU DO AT THIS POINT

Read all 6 requirements. If something is missing or wrong, tell Kiro in chat. For example: "REQ-3 should also track waist measurements." Kiro will update the requirement. Only approve when everything looks right. You cannot easily change requirements later without affecting the design and code.

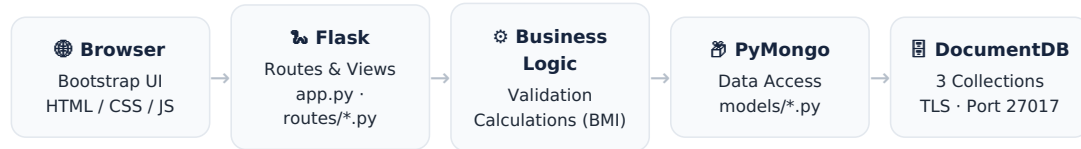
## 2 Step 2 — Architecture & DocumentDB Schema

After requirements are approved, Kiro designs the full system — automatically

Once you approve the requirements, Kiro moves to the design phase. It creates a complete technical architecture document including system design, database schema, data flow diagrams, and component interfaces. You review and approve this too before any code is written.

### Application Architecture (MVC Pattern)

SYSTEM ARCHITECTURE — BROWSER TO DOCUMENTDB



#### 🔍 MVC PATTERN EXPLAINED

**Model (M):** The data layer — DocumentDB collections + PyMongo models. This is where data lives.

**View (V):** The HTML pages (Bootstrap templates) — what the user sees.

**Controller (C):** Flask routes — they receive requests, ask the Model for data, and send it to the View.

### DocumentDB Collections — The Database Schema

DocumentDB stores data in collections (like tables in SQL), and each entry is a document (a JSON object). Here's exactly what gets stored:

#### COLLECTION 1: CLIENTS — one document per gym client

```
{
  "_id": ObjectId("64a8f..."),
  "name": "Ahmed Al Rashid",
  "email": "ahmed@email.com",
  "phone": "+971-50-1234567",
  "date_of_birth": "1990-05-15",
  "goals": "Lose 10kg, build core strength",
  "health_notes": "Lower back injury — avoid deadlifts",
  "created_at": "2025-06-01T09:00:00Z"
}
```

## COLLECTION 2: WORKOUT\_PROGRAMS — nested arrays (this is why DocumentDB is perfect!)

```
// In SQL this would need 3 tables + JOINS
{
  "_id": ObjectId("64b9a..."),
  "client_id": ObjectId("64a8f..."), // links to client
  "name": "Week 1 — Full Body",
  "description": "Beginner strength program",
  "exercises": [ // array of exercises!
    {
      "name": "Squat",
      "sets": 3,
      "reps": 12,
      "weight_kg": 40,
      "rest_seconds": 90
    },
    {
      "name": "Push-up",
      "sets": 3,
      "reps": 15,
      "weight_kg": 0, // bodyweight
      "rest_seconds": 60
    }
  ]
}
```

## COLLECTION 3: BODY\_MEASUREMENTS — one document per measurement session

```
{
  "client_id": ObjectId("64a8f..."),
  "date": "2025-06-15",
  "weight_kg": 82.5,
  "height_cm": 175,
  "bmi": 26.9, // auto-calculated
  "body_fat_pct": 18.5,
  "notes": "Down 2kg from last month!"
}
```

### 💡 WHY IS NESTING EXERCISES INSIDE WORKOUT\_PROGRAMS SO POWERFUL?

In a SQL database, you'd have a `workout_programs` table and a separate `exercises` table, linked by an ID. To get a full workout, you'd need a JOIN query. In DocumentDB, all exercises live inside the workout document — one query gets everything. Simpler, faster, more natural.

### 3 Step 3 — Implementation Planning (MVP Mode)

Kiro converts the approved design into a numbered task list — you choose full or MVP

After you approve the architecture design, Kiro creates an implementation plan. By default it generates 15 main tasks with 22+ sub-tasks — a full production app. But for today's workshop, we ask for the MVP version.

#### The MVP Prompt

KIRO CHAT INPUT — MVP MODE

```
create MVP.  
Please just focus on core functionalities.  
It is MVP for application.  
  
Use below documentdb connection string:  
mongodb://adminuser:<password>@127.0.0.1:27017/?tls=true  
&tlsCAFile=global-bundle.pem&retryWrites=false  
&tlsAllowInvalidHostnames=true&directConnection=true
```

#### ? WHY DO WE GIVE THE CONNECTION STRING AT THIS POINT?

Kiro uses Test-Driven Development — it runs real tests against your real database after each task. So it needs the actual connection string to connect. The 127.0.0.1:27017 is your local machine — which forwards to DocumentDB via the SSH tunnel you set up.

#### Full Plan vs MVP Plan

##### ✗ FULL PLAN (SKIP FOR TODAY)

15 main tasks  
22+ sub-tasks  
All features including reporting, advanced auth, API  
Days to build properly

##### ✓ MVP PLAN (WHAT WE BUILD TODAY)

8 focused tasks  
Core features only  
Fully working web app  
Under 2 hours

#### The 8 MVP Tasks — What Each One Builds

##### 1 Project Structure & Config

Creates the folder structure: app/, models/, routes/, templates/, static/. Sets up Flask app factory pattern, config file with DocumentDB settings, environment variables.

##### 2 DocumentDB Connection Implementation

Writes the PyMongo connection code using your connection string + TLS certificate. Includes connection pooling, error handling, health check endpoint. Runs a test to verify the actual connection works.

##### 3 Client Management Models & Forms

Creates the Client model with CRUD operations (Create, Read, Update, Delete). Builds the client registration form with validation. Adds search and filter functionality. Tests all CRUD operations against DocumentDB.



#### 4 Workout Program Management System

Creates the WorkoutProgram model with nested exercise arrays. Builds forms to add/edit programs and individual exercises. Links programs to clients. Tests creation and retrieval of nested structures.

#### 5 Body Measurement Tracking with Trends

Creates the Measurement model. Auto-calculates BMI from weight and height. Detects significant changes ( $\pm 5\%$ ). Builds chart-ready data arrays for trend visualization.

#### 6 Progress Dashboard

Uses DocumentDB's aggregation pipeline — like SQL GROUP BY on steroids — to calculate summary stats: total clients, active programs, average BMI, goal completion rates. Displays charts and trend lines.

#### 7 Basic Authentication

Login/logout with session management. Two roles: Trainer (can view/edit their own clients) and Admin (can see all clients, all trainers). Password hashing (never stored in plain text).

#### 8 Web Interface (Bootstrap)

Responsive UI using Bootstrap — works on desktop and mobile. Navigation bar, sidebar, data tables, forms with validation feedback, charts using Chart.js, modal dialogs.

## 4 Step 4 — Build, Run & The Live App

Kiro executes all 8 tasks automatically — you verify, then run the app

### How Kiro Executes Tasks

1 Kiro starts Task 1 — writes all the code for project structure

2 Kiro runs tests automatically — verifies the code works as expected

3 Kiro asks you to verify — "Does this look right? Can you confirm the folder structure?" You check and approve.

4 Kiro moves to Task 2 — repeats the same cycle for each of the 8 tasks

✓ All 8 tasks complete — run `python run.py` and open your browser

#### ⚠ NEVER SKIP KIRO'S VERIFICATION REQUESTS!

When Kiro says "please verify this works" — actually check it. If Task 2 (DB connection) is broken and you skip verification, Tasks 3–8 will all fail on top of it. Five minutes of verification saves hours of debugging.

The Finished App — What You'll See

MAIN DASHBOARD — <http://localhost:5000/dashboard>

 **FitCenter Pro Dashboard**

Trainer: Hani | [Logout](#)

OVERVIEW — JUNE 2025

24

Total Clients

18

Active Programs

73%

Goal Achievement

| CLIENT          | LAST SESSION | BMI  | PROGRESS        | ACTION               |
|-----------------|--------------|------|-----------------|----------------------|
| Ahmed Al Rashid | 2025-06-14   | 26.9 | On track        | <a href="#">View</a> |
| Sara Mohammed   | 2025-06-12   | 22.1 | Ahead           | <a href="#">View</a> |
| Khalid Hassan   | 2025-06-10   | 31.2 | Needs attention | <a href="#">View</a> |

CLIENT REGISTRATION FORM — <http://localhost:5000/clients/new>

Add New Client

Full Name

\* Ahmed Al Rashid

Email Address

\* ahmed@email.com

Phone

+971-50-1234567

Date of Birth

1990-05-15

Fitness Goals

Lose 10kg, build core strength

Health Notes

Lower back injury — avoid deadlifts

WORKOUT PROGRAM BUILDER — <http://localhost:5000/workouts/new>

Create Workout Program — Week 1 — Full Body Strength

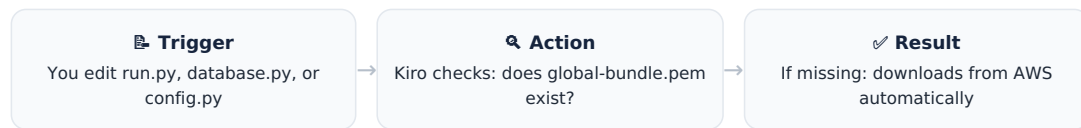
| # | EXERCISE     | SETS | REPS | WEIGHT (KG) | REST (SEC) |
|---|--------------|------|------|-------------|------------|
| 1 | Squat        | 3    | 12   | 40          | 90         |
| 2 | Push-up      | 3    | 15   | 0 (BW)      | 60         |
| 3 | Dumbbell Row | 4    | 10   | 15          | 75         |

# ⚡ Kiro Hooks — Workflow Automation

Hooks are like mini GitHub Actions that live inside your IDE

Hooks are one of Kiro's most powerful features. They are automated actions triggered by file events. When a specific file is edited or saved, Kiro automatically runs a script or action. You never have to remember to do it manually.

## Example Hook: Auto-Download SSL Certificate



Without this hook, every new team member who clones the repo forgets to download the certificate and spends 20 minutes debugging why the connection fails. The hook makes this impossible to forget.

## How a Hook is Defined (in Kiro)

KIRO HOOK DEFINITION — .KIRO/HOOKS/SSL-CERT.YAML

```
name: Download SSL Certificate
trigger:
  type: file_edit
  files:
    - run.py
    - database.py
    - config.py
action:
  type: shell
  command: |
    if [ ! -f global-bundle.pem ]; then
      wget https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem
      echo "✓ SSL certificate downloaded!"
    fi
```

## Other Useful Hooks You Could Add

### 🔍 AUTO-LINT ON SAVE

**Trigger:** Any .py file saved

**Action:** Run flake8 and show errors in terminal

**Benefit:** Catch code style issues immediately, not at code review

### 📦 UPDATE REQUIREMENTS.TXT

**Trigger:** Any .py file that adds a new import

**Action:** Run pip freeze > requirements.txt

**Benefit:** requirements.txt is always up to date, no manual work

### 🌱 SEED TEST DATA

**Trigger:** database.py or models/ edited

**Action:** Run seed script to populate DocumentDB with sample data

**Benefit:** Always have data to test against after DB changes

### 📖 GENERATE API DOCS

**Trigger:** Any file in routes/ is edited

**Action:** Auto-generate API documentation (Swagger/OpenAPI)

**Benefit:** Docs are always in sync with code

## What's Next — Extending the MVP

The MVP is just the foundation — here's how to grow it

### Mobile App Integration

Add REST API endpoints so any mobile app (React Native, Flutter) can connect to the same DocumentDB backend.

*Ask Kiro: "Scaffold a FastAPI REST wrapper for the existing Flask app"*

### ML-Powered Analytics

Feed DocumentDB aggregation data into Amazon SageMaker to predict whether a client will reach their goal.

*DocumentDB → Python pipeline → SageMaker model → prediction displayed in dashboard*

### Session Scheduling

Add a booking system for trainer-client sessions with calendar integration, reminders, and cancellation handling.

*Ask Kiro: "Add a new spec for session scheduling with Google Calendar integration"*

### Nutrition Tracking

Extend the DocumentDB schema with meal plans and calorie tracking — no migration needed, just add new fields to documents.

*This is where DocumentDB shines — add fields to existing documents without breaking anything*

### Wearable Integration

Sync data from Garmin, Apple Watch, Fitbit via their APIs. Webhooks push new data directly into DocumentDB.

*Ask Kiro: "Add webhook endpoints for Garmin Connect and Apple Health data"*

### Production-Ready Security

Replace hardcoded credentials with AWS Secrets Manager. Replace basic auth with Amazon Cognito. Deploy on ECS or Lambda.

*AWS Secrets Manager + Amazon Cognito + ECS/Lambda = enterprise-grade security*

# Quick Reference Cheat Sheet

Everything you need in one place — save this page!

## Essential Commands

### TERMINAL

```
# Install dependencies
pip install flask pymongo dnspython

# Download SSL certificate
wget https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem

# Open SSH Tunnel (leave running!)
ssh -L 27017:YOUR-DOCDB-ENDPOINT:27017 \
    ec2-user@YOUR-EC2-BASTION -N

# Run the app
python run.py
```

## DocumentDB Connection String

### CONNECTION STRING

```
mongodb://adminuser:<password>@127.0.0.1:27017/
?tls=true
&tlsCAFile=global-bundle.pem
&retryWrites=false
&tlsAllowInvalidHostnames=true
&directConnection=true
```

## Kiro Workflow Summary

| PHASE | WHAT HAPPENS                                           | YOUR ACTION |
|-------|--------------------------------------------------------|-------------|
| 1     | Kiro reads prompt, generates EARS requirements         | Review      |
| 2     | Kiro creates full system architecture + DB schema      | Review      |
| 3     | Kiro creates 8-task MVP plan, needs connection string  | Provide     |
| 4     | Kiro executes tasks, runs tests, asks for verification | Verify ✓    |
| 5     | App is built — run with python run.py                  | Done! 🎉     |

Troubleshooting

| ERROR                | FIX                                                   |
|----------------------|-------------------------------------------------------|
| Connection refused   | SSH tunnel not running — open it                      |
| SSL handshake failed | global-bundle.pem missing — download it               |
| Auth failed          | Wrong password in connection string                   |
| Port already in use  | Kill old tunnel: <code>pkill -f "ssh -L 27017"</code> |

Key Links

 Kiro IDE: [kiro.dev/downloads](https://kiro.dev/downloads)

 Kiro Docs: [kiro.dev/docs](https://kiro.dev/docs)

 DocumentDB Docs: [docs.aws.amazon.com/documentdb](https://docs.aws.amazon.com/documentdb)

 AWS Blog (original): [aws.amazon.com/blogs/database/](https://aws.amazon.com/blogs/database/)

 Cost estimate: [calculator.aws](https://calculator.aws)

The 3 Golden Rules for This Workshop



**Always Verify**

When Kiro asks you to verify, actually do it. Don't skip. Five minutes now saves hours later.



**Approve Before Moving**

Requirements → approve. Design → approve. Plan → approve. Never rush ahead.



**Keep Tunnel Open**

The SSH tunnel terminal must stay open the entire time. If the app disconnects, check if it's still running.

Workshop Resources

[kiro.dev](https://kiro.dev)  
[kiro.dev/docs](https://kiro.dev/docs)  
[kiro.dev/pricing](https://kiro.dev/pricing)

AWS References

[Amazon DocumentDB](#)  
[DocumentDB Docs](#)  
[AWS Pricing Calculator](#)

Official Blog Post

By Anshu Vajpayee & Karthik Vijayraghavan  
Published: December 11, 2025  
[aws.amazon.com/blogs/database/...](https://aws.amazon.com/blogs/database/)